PREVIOUS GREATER ELEMENT

Using Pointers and Nested Loops

Name : Rudra Pratap Singh

SAP ID : 590032811

Batch : 16

1. Problem Statement

Write a C program that reads the size and the elements of a variable-length integer array arr[]. The array must be passed to a function named previous_greater() using its address, and pointers must be used to access the array in both the calling function and previous_greater().

For every element of the array, the function must find the previous greater element, i.e. the nearest element to its left that is strictly greater than it. If no such element exists, -1 must be displayed. The results must be printed in the order of the array elements, separated by commas, using a nested-loop approach.

2. Aim

To find the previous greater element of every element in an integer array using pointers, dynamic memory allocation, functions and nested loops in C.

3. Concepts Used

- **Pointers:** the array is passed as int *arr and its elements are accessed with pointer arithmetic, *(arr + i).
- **Dynamic memory allocation:** malloc() creates an array whose size is decided at run time; free() releases it.
- **Functions:** the logic is separated into previous_greater(); main() handles input.
- **Nested loops:** the outer loop picks each element; the inner loop scans the elements to its left.
- **Input validation:** the return value of scanf() is checked and the size must be positive.

4. Algorithm

1. Start.
2. Read the size n of the array and check that n > 0.
3. Allocate memory for n integers using malloc() and read the n elements.
4. Call previous_greater(arr, n), passing the base address of the array.
5. For each index i from 0 to n - 1, set result = -1.
6. For each index j from i - 1 down to 0 (nearest element first): if *(arr + j) > *(arr + i), set result = *(arr + j) and stop the inner loop.
7. Print result, followed by a comma and a space unless it is the last element.
8. Free the allocated memory.
9. Stop.

5. Source Code

```
/*
 * Program : Previous Greater Element
 * Name    : Rudra Pratap Singh
 * SAP ID  : 590032811
 * Batch   : 16
 *
 * Description:
 *   Reads the size and elements of an array, then finds for every element
 *   the nearest greater element on its left. Prints -1 if none exists.
 */

#include <stdio.h>
#include <stdlib.h>

/* Prints the previous greater element of every element in arr[] */
void previous_greater(int *arr, int n)
{
    int i, j;
    int result;

    for (i = 0; i < n; i++)
    {
        result = -1;                        /* assume no greater element */

        /* scan leftwards, starting from the nearest element */
        for (j = i - 1; j >= 0; j--)
        {
            if (*(arr + j) > *(arr + i))    /* first greater element found */
            {
                result = *(arr + j);
                break;                      /* nearest one, so stop here */
            }
        }

        if (i == n - 1)
            printf("%d", result);
        else
            printf("%d, ", result);
    }
    printf("\n");
}

int main(void)
{
    int n, i;
    int *arr;

    printf("Enter the size of the array: ");
    if (scanf("%d", &n) != 1 || n <= 0)
    {
        printf("Invalid size. Size must be a positive integer.\n");
        return 1;
    }

    arr = (int *)malloc(n * sizeof(int));  /* variable-length array */
    if (arr == NULL)
    {
        printf("Memory allocation failed.\n");
```

```
        return 1;
    }

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
    {
        if (scanf("%d", arr + i) != 1)
        {
            printf("Invalid input.\n");
            free(arr);
            return 1;
        }
    }

    printf("Output:\n");
    previous_greater(arr, n);              /* pass array using its address */

    free(arr);
    return 0;
}
```

6. Working of the Program

main() reads n, allocates the array with malloc() and fills it using the pointer arr + i. It then passes the array to previous_greater() by its address.

previous_greater() receives the address in the pointer int *arr. The outer loop (i) selects the current element. The inner loop (j) starts at i - 1 and moves left, so the first element found that is greater than *(arr + i) is automatically the nearest one; break then ends the search. If the inner loop finishes without a match, result stays -1.

Since the first element has no elements on its left, its answer is always -1.

7. Dry Run

Consider the array 13 7 6 12. The inner loop scans from right to left:

i	arr[i]	Elements checked (right to left)	Result
0	13	None (no element on the left)	-1
1	7	13 is greater than 7	13
2	6	7 is greater than 6	7
3	12	6 (no), 7 (no), 13 (yes)	13

Final output: -1, 13, 7, 13

8. Sample Input and Output

The program was compiled with gcc -Wall -Wextra (no warnings) and tested on the following inputs.

Test	Input Array	Output
1	4 5 2 25	-1, -1, 5, -1
2	13 7 6 12	-1, 13, 7, 13
3	1 3 2 4	-1, -1, 3, -1
4	1 2 3 4 5	-1, -1, -1, -1, -1
5	9 9 3 9 1 10	-1, -1, 9, -1, 9, -1
6	7 (single element)	-1

Sample run (Test 2):

```
Enter the size of the array: 4
Enter 4 elements: 13 7 6 12
Output:
-1, 13, 7, 13
```

Invalid size handling:

```
Enter the size of the array: 0
Invalid size. Size must be a positive integer.
```

9. Complexity Analysis

- **Time complexity:** $O(n^2)$ in the worst case, because for each of the n elements the inner loop may scan up to all elements on its left (e.g. a strictly increasing array).
- **Space complexity:** O(1) extra space, apart from the input array itself.

10. Conclusion

The program successfully finds the previous greater element for every element of a variable-length array. It meets all the requirements: the array is passed using its address, pointers are used to access it in both functions, the nearest greater element on the left is selected, -1 is printed when none exists, the results are comma-separated in array order, and a nested-loop approach is used. All the given examples produce the expected output.